

1 P4 C5: certified range and certificate checklist

Task source: [The certified boundary](#), checked on 2026-09-26. This package is not a platform submission or an organizer acceptance.

1.1 Claimed certified boundary: 14

C5 is solved for the contiguous claimed range 2–14, the largest range for which this package currently completes every C5 benchmark, including an observed rule-on/rule-off comparison. This endpoint describes our completed certificate, not the largest true size of the conjecture.

A complete written proof for sizes 2–8 appears below; the finite reduction and exhaustive certificates then handle sizes 9–14 in order. Lean independently proves 2–12. For 13, assume it is the least failing size, normalize a minimum-modulus-sum counterexample, and apply the [proved finite reduction and pruning rules](#). The complete modulus search leaves exactly the explicit thirteen-list; its seven-entry obstruction is impossible, as proved in the [C4 argument](#). Thus 13 holds. Repeating the least-counterexample argument at 14 gives an empty enumerated list, proving 14. Every smaller-size dependency is discharged in order.

The full computational replay for this C5 claim took **16.109 seconds**, including both independent implementations, exact node-count and survivor comparisons, all rule-on/off runs at 9–14, positive control, and the saved residue-certificate checks. Reproduce with:

```
python3 -B cagent/p4/audit/replay.py --max-k 14
```

It writes outputs to a new temporary directory by default, preserving bundled evidence. [Recorded replay](#) contains every size’s nodes, wall clock, exact baseline agreement, and equal final lists with early strong rules enabled and disabled. Counts agree exactly; elapsed time is machine-dependent.

Computational evidence also extends the mathematical argument through 16, but its additional C5 rule-off runs are unfinished. Those sizes are **not included in this C5 boundary claim**. No completed exhaustive search at 17 or above is claimed.

1.2 Isolated sizes

[Known-cases proof](#) reconstructs, line by line, the exclusions of **24 and 30 as least counterexample sizes**. It explicitly assumes all smaller sizes for each exclusion. It uses finite prime-support counting, not a search, and repairs an insufficient final inference in the source’s exceptional k=30 argument. These are not unconditional 24-class or 30-class theorems inferred from our range through 16.

1.3 Nonvacuous control

The unchanged base enumeration function accepts a gcd cap and a target length. Run `enumerate.py 7 --target-length 6` without the least-counterexample-only strong flag. It returns six copies of modulus 6, witnessed by residues `[0,1,2,3,4,5]`. These classes are pairwise disjoint; every pair gcd is 6, below the requested cap 7. The density is exactly one. The control uses the same finite domain, anchor enumeration, compatibility checks, density tests, and branching code as ordinary runs; it does not inject a preselected answer or disable a rejection because it happens to reject the desired example. The different cap and length

are explicit inputs. Least-counterexample-only conditions cannot be imposed when those parameters differ, and the implementation rejects that invalid strong-mode combination.

The data are in [positive-control.jsonl](#), and fresh replay regenerates them. The control proves the machinery can return admissible objects; it is not a counterexample to the conjecture, whose cap and target size are equal.

1.4 Complete survivor decisions and smallest obstructions

Within the C5 claim, at 9–12 and 14 the final modulus survivor lists are empty; at 13 there is one reduced list. The additional, separately scoped work at 15 and 16 has 30 and 77 reduced lists. Every list is written out in full in the linked JSONL data, not represented only by a count. The [C4 solution](#) and [audit report](#) identify the files and the distinction between raw density and reduced lists.

The 13-list has a seven-entry obstruction, with concrete realizations of every smaller submultiset. Its impossibility and minimum cardinality are also kernel-checked in Lean. For each of the other 107 lists, `global-minima-claims.jsonl` selects an obstruction, `minimum-refutations.jsonl` excludes all its residue assignments, and `global-minima-facts.jsonl` supplies actual residues for every smaller submultiset. The replay verifies inclusion, cardinality, every CRT inequality, full coverage of smaller submultisets, and all branches of each obstruction tree. This proves minimum cardinality inside each individual survivor, not merely that deleting one position repairs a chosen obstruction.

1.5 Independent method and pruning comparison

The first enumeration grows nondecreasing occurrence lists after selecting an anchor triple with replacement. The second builds cliques of distinct compatible modulus values, then chooses positive multiplicities. Their divisor-generation routines also differ. Exact final lists are compared elementwise at every enumerated size. Different traversal node counts are expected; differences in final lists are not silently reconciled. The [reduction proof](#) explains every mathematical pruning rule and every capacity or early-future check.

All numerical rules are necessary conditions proved in that document; an unfinished run is never interpreted as emptiness. For every computational size in the C5 claim, 9–14, completed runs with all three early strong rules enabled and disabled have identical final reduced survivor lists. The recorded comparisons turn off early feasibility checks while retaining the same proved complete-tuple conditions: they compare the same mathematical output set. Sizes 2–8 use complete direct Lean proofs, so there is no computational pruning or survivor list to compare there.

1.6 Written base cases 2–8

The following elementary proof supplies every base case needed for the least-counterexample induction. It is the written C3 argument, included here so the C5 range does not depend solely on a linked theorem.

Let $A_i = a_i \pmod{m_i}$ be k pairwise disjoint congruence classes of integers, where every m_i is a positive integer. Repeated moduli are permitted. We claim that some two of their moduli satisfy

$$\gcd(m_i, m_j) \geq k.$$

The elementary intersection criterion is

$$(a_i \pmod{m_i}) \cap (a_j \pmod{m_j}) \neq \emptyset \iff \gcd(m_i, m_j) \mid (a_j - a_i). \quad (\text{B1})$$

For the reverse implication, Bézout gives integers u, v with $g = um_i + vm_j$ for $g = \gcd(m_i, m_j)$. If $a_j - a_i = tg$, then $z = a_i + tum_i = a_j - tvm_j$ lies in both classes. The forward implication follows because an integer in both classes makes $a_j - a_i$ an integer linear combination of m_i, m_j . Thus disjointness says that a_i and a_j are *different modulo their modulus gcd*. We will use the following consequence: if $g_{ij} = \gcd(m_i, m_j)$ divides D , then $a_i \not\equiv a_j \pmod{D}$; equality modulo D would imply equality modulo g_{ij} .

Suppose, toward a contradiction for any fixed k , that every pair gcd is less than k . It cannot be 1, by (B1), so throughout that case

$$2 \leq g_{ij} < k \quad (i \neq j). \quad (\text{B2})$$

1.7 The cases $k = 2, 3, 4$

For $k = 2$, the sole gcd cannot be 1, proving the claim. For $k = 3$, (B2) makes every pair gcd equal to 2. Three residues would then be pairwise different modulo 2, an impossibility.

For $k = 4$, every pair gcd is 2 or 3. If all moduli are even, each pair gcd is 2, so already three residues would be distinct modulo 2. Otherwise choose an odd modulus. Its gcd with each other modulus must be 3, making every modulus divisible by 3. All pair gcds are then 3, forcing four distinct residues modulo 3. Both alternatives are impossible.

1.8 The case $k = 5$

Here every pair gcd lies in $\{2, 3, 4\}$. If all five moduli are even, each pair gcd is 2 or 4 and hence divides 4. The five residues must be pairwise distinct modulo 4, impossible.

Otherwise choose an odd modulus m_i . Its gcd with any other modulus is odd and in $\{2, 3, 4\}$, so equals 3. Consequently all five moduli are divisible by 3. Every pair gcd is now divisible by 3 and in $\{2, 3, 4\}$, so equals 3. Five pairwise distinct residues modulo 3 are impossible.

1.9 The case $k = 6$

Every pair gcd is in $\{2, 3, 4, 5\}$, so any two moduli share at least one prime from $P = \{2, 3, 5\}$. None of these three primes divides *all* six moduli. If 2 did, every pair gcd would be 2 or 4, forcing six pairwise distinct residues modulo 4. If 3 did, every pair gcd would equal 3, forcing six distinct residues modulo 3. If 5 did, every pair gcd would equal 5, forcing six distinct residues modulo 5. Each conclusion is impossible.

Every modulus is divisible by at least two primes from P . Indeed, it is divisible by at least one because it shares a prime from P with every other modulus. If it were divisible by just one, say p , every other modulus would have to share p with it, making p common to all six.

There are only three two-prime types, $\{2, 3\}$, $\{2, 5\}$, and $\{3, 5\}$. If two moduli contained the same type, their gcd would be at least the product of its primes, hence at least 6, contradicting (B2). Assign to each modulus any one of its contained two-prime types. The assignment is injective, so six moduli cannot be assigned to three types.

1.10 The case $k = 7$

Now every pair gcd is in $\{2, 3, 4, 5, 6\}$, and every pair shares at least one prime in $P = \{2, 3, 5\}$. Again no prime in P divides all seven moduli. If 3 did, each gcd would be 3 or 6 and hence divide 6; seven residues could not be pairwise distinct modulo 6. If 5 did, each gcd would equal 5; already six residues could not be pairwise distinct modulo 5.

If 2 divided all seven moduli, among the seven residues at least four would have the same parity $r \in \{0, 1\}$. Restrict to those four classes and write their residues and moduli as $a_s = 2b_s + r$ and $m_s = 2n_s$. The four reduced classes $b_s \pmod{n_s}$ are pairwise disjoint: an integer common to two reduced classes would give, after multiplying by 2 and adding r , an integer common to the corresponding original classes. The established $k = 4$ case therefore supplies a pair with $\gcd(n_s, n_t) \geq 4$. Their original gcd is $2 \gcd(n_s, n_t) \geq 8$, contrary to (B2). Hence 2 is not common either.

As at $k = 6$, each modulus must contain at least two primes from P : it contains one because it shares a prime from P with each other modulus, and containing only one would make that prime common to all seven. Choose an index i for which $2 \nmid m_i$ and an index j for which $3 \nmid m_j$. Then $3 \cdot 5 \mid m_i$ and $2 \cdot 5 \mid m_j$, so $i \neq j$.

Consider any of the other five indices t . If $5 \mid m_t$, it also contains 2 or 3. In the first case $10 \mid \gcd(m_t, m_j)$; in the second $15 \mid \gcd(m_t, m_i)$. Either contradicts (B2). Therefore $5 \nmid m_t$, and the two-prime property forces $2 \cdot 3 \mid m_t$.

For distinct indices s, t among those five, $6 \mid \gcd(m_s, m_t)$, so (B2) makes their gcd exactly 6. For each such t , the gcd of m_t and m_i is divisible by 3, is not divisible by 2 because m_i is odd, and is at most 6; it is therefore exactly 3. Their five residues must consequently be pairwise distinct modulo 6 while all avoiding $a_i \pmod{3}$. Of the six classes modulo 6, exactly four avoid one prescribed class modulo 3. Five distinct choices from four are impossible.

1.11 The case $k = 8$

Here every pair gcd belongs to $\{2, 3, 4, 5, 6, 7\}$. Remove any one index. The remaining seven classes are still disjoint, so the proved $k = 7$ case gives a pair whose gcd is at least 7. By (B2), that gcd is exactly 7. Hence, for every removed index, two of the retained moduli are divisible by 7.

Let S be the set of indices whose moduli are divisible by 7. The preceding conclusion gives $|S| \geq 2$; in fact $|S| \geq 3$, because if it had exactly two members, removing either one would leave no possible pair with gcd 7. On the other hand, S is not all eight indices. If every modulus were divisible by 7, (B2) would make every pair gcd equal to 7, forcing eight distinct residues modulo 7.

Choose an index $t \notin S$. For every $s \in S$, the gcd of m_s and m_t is in $\{2, 3, 4, 5, 6\}$. It is therefore divisible by one of 2, 3, 5, which divides both moduli. Distinct $s_1, s_2 \in S$ cannot share any of these three primes, because 7 and that prime would both divide $\gcd(m_{s_1}, m_{s_2})$, making it at least 14, contrary to (B2). Thus each member of S requires a *different* prime from $\{2, 3, 5\}$ to intersect m_t in this divisibility sense. It follows that $|S| \leq 3$, hence $|S| = 3$.

There are therefore five indices outside S . The same argument applies to *every* outside index: its modulus is divisible by all three primes 2, 3, and 5, because it must share one distinct such prime with each of the three moduli in S . Any two outside moduli then have gcd at least $2 \cdot 3 \cdot 5 = 30$, contradicting (B2). This completes the proof through eight.

The bound is sharp at each k : the k classes $0, 1, \dots, k-1 \pmod{k}$ are pairwise disjoint, and

every pair of moduli has gcd exactly k .

1.12 Per-size replay measurements

The complete boundary-14 replay took **16.109 seconds** in the recorded laptop run, including search, comparisons, the positive control, and certificate checks. These are observed wall times, not portable time limits. The exact node counts and output lists match the saved baselines. Both implementations report **complete: true** for every row. The rule-on and rule-off columns refer to the occurrence enumerator; all three optional early feasibility rules were toggled together while the proved complete-tuple checks stayed in force.

k	Raw lists	Final reduced lists	Tuple nodes off/on	Clique nodes off	Tuple wall seconds off/on	Clique wall seconds off
9	0	0	10 / 10	20	0.036 / 0.037	0.036
10	0	0	92 / 92	85	0.036 / 0.038	0.036
11	0	0	9,541 / 6,222	7,460	0.115 / 0.111	0.114
12	0	0	773 / 53	699	0.048 / 0.048	0.042
13	51	1	347,628 / 143,111	217,024	5.576 / 2.489	4.010
14	0	0	12,884 / 322	8,359	0.160 / 0.104	0.090

At each size the exact final lists are identical with the early rules on and off. Each optional rule only removes search branches, and the complete-tuple decision is unchanged. A run with any subset of these rules therefore has a final list between the all-on and all-off lists; because those endpoint lists coincide, switching off any one rule also leaves the final list unchanged. The two unpruned methods also have identical raw lists elementwise. Thus the exact final list is empty at 9–12 and 14, while at 13 it contains only [10, 10, 10, 10, 10, 12, 12, 12, 12, 24, 36, 40, 45]. Their traversal node counts differ because one recurses on occurrences and the other on distinct-value cliques; no output disagreement was reconciled silently. The complete lists and node-depth counters appear in the [tuple output](#) and [clique output](#), and the on/off output is [recorded separately](#). The [replay file](#) verifies exact baseline node counts, survivor arrays, positive control, and residue certificates. Run `python3 -B cagent/p4/audit/replay.py --max-k 14` from the repository root; it writes a fresh temporary output directory.

1.13 Closing the contiguous induction through 14

Assume the result has been proved for every size from 2 through $k - 1$, where $9 \leq k \leq 14$, and suppose that a k -class counterexample exists. Choose one with minimum modulus sum. Appendix A proves, from precisely these smaller-size premises, that its modulus multiset lies in the finite domain and survives every necessary test implemented by both searches. The complete outputs at $k = 9, 10, 11, 12$ and 14 have **no** final reduced multiset, so each of those steps is impossible.

At $k = 13$, the unique final reduced multiset is $[10, 10, 10, 10, 10, 12, 12, 12, 12, 24, 36, 40, 45]$. Its submultiset $[10, 10, 10, 10, 10, 12, 45]$ cannot have pairwise disjoint residues. The gcd of 10 and 12 is 2, so all five 10-class residues have parity opposite the 12-class. Because the five 10-classes are pairwise disjoint, their residues are distinct modulo 10 and exhaust the five values of that parity. Their reductions modulo 5 are therefore all five possible residues. But the 45-class has gcd 5 with every 10-class, so it intersects one of them by the intersection criterion, a contradiction. The saved direct audit gives valid residues for every submultiset of the complete 13-list with fewer than seven entries; hence this seven-entry obstruction is minimum in that survivor. The impossibility and minimum cardinality are also kernel-checked in the pinned `Survivor13` and `Minimality13` modules linked above.

Starting with the written base cases 2–8 and applying these six steps in increasing order proves the assertion for every $2 \leq k \leq 14$. The certificates at 15 and 16 are additional evidence only; their unfinished early-rule-off benchmark is not used in the claimed C5 boundary.

2 Appendix A — Complete finite reduction and pruning proof

This note proves the mathematical implications used by `enumerate.py`, `clique.py`, and the modulus and residue checks in `audit.py`. The source is Kevin O’Bryant, *On Z.-W. Sun’s Disjoint Congruence Classes Conjecture*, arXiv:math/0604347v2, especially Proposition 2 and Lemmas 5–6. The arguments are reconstructed below rather than delegated to that citation. They apply to a **least-size** counterexample, and the sum-minimal normalization applies after choosing, among counterexamples of that size, one minimizing the sum of the moduli. They do not by themselves prove a conjecture at a size whose smaller cases are unsettled. This document establishes pruning soundness; completeness of a particular finite run additionally requires its recorded `complete: true` result and replay of its output.

2.1 Starting hypothesis and finite normalization

Assume $k \geq 4$ is the least size of pairwise disjoint integer congruence classes $a_i \pmod{m_i}$, with positive moduli, for which every $\gcd(m_i, m_j) < k$. Choose such a system with minimal $\sum_i m_i$. By the two-class Chinese remainder theorem, classes $a_i \pmod{m_i}$ and $a_j \pmod{m_j}$ meet if and only if $\gcd(m_i, m_j) \mid a_i - a_j$. Thus all pair gcds are in $\{2, \dots, k-1\}$; a gcd of 1 cannot occur in a disjoint system. This is the paper’s [Proposition 2](#).

Let

$$N = \text{lcm}_{i < j} \gcd(m_i, m_j), \quad L_k = \text{lcm}(1, 2, \dots, k-1).$$

Certainly $N \mid L_k$. If a maximal prime-power factor p^e of some m_i did not divide N , every other m_j would have p -adic valuation below e : otherwise $p^e \mid \gcd(m_i, m_j) \mid N$. Dividing just m_i by p consequently leaves every $\gcd(m_i, m_j)$ unchanged, since its p -valuation was already less than e . All disjointness inequalities remain true by the Chinese remainder criterion, while the modulus sum decreases. This contradicts its choice. Every maximal prime-power factor of every m_i therefore divides N , so $m_i \mid N \mid L_k$. Also $N = \text{lcm}_i m_i$: every pair gcd divides its participating moduli, and every modulus divides N . This proves the finite reduction in [Lemma 6\(1\)](#), pp. 3–4.

Every prime divisor of any m_i is therefore a prime below k . A modulus 1 is impossible because all its pair gcds would be 1. No modulus is a prime power. To prove the latter, suppose $m_i = p^e$. Every modulus is divisible by p , since it has gcd greater than 1 with m_i ; also $p < k$. Among the k residues, choose $t = \lceil k/p \rceil$ with one common residue modulo p , where $2 \leq t < k$. Subtract

that common residue and divide these selected residues and moduli by p . For every selected pair, both its residue difference and pair gcd are divided by p , hence the divided classes remain disjoint. Minimality of k forces a pair gcd at least t among them, making the original pair gcd at least $pt \geq k$, a contradiction. This proves [Lemma 6\(4\)](#), pp. 4–5. Thus a complete finite *candidate* domain for modulus **values** is the set of divisors of L_k having at least two distinct prime factors; later conditions may exclude individual values or combinations. Both `enumerate.domain` (prime-power product expansion) and `clique.divisors_with_two_primes` (divisor generation and trial division) compute that set; repetitions remain possible.

For every prime power $q \mid m_i$, also $q \mid N$. Since N is the lcm of pair gcds, some pair gcd is divisible by q , so at least two moduli are divisible by q . Whether or not i belongs to that pair, at least one of those two moduli is different from i . This proves the *prime-power support* rule, [Lemma 6\(3\)](#). The code tests the maximal power of each prime in each modulus; testing these powers suffices because every lower power is then supported as well. `enumerate.power_support`, `audit.maximal_power_failure`, and `clique.extra_good` all implement this implication on complete multisets.

2.2 Anchor counts

Call a modulus an anchor when $k - 1 \mid m_i$. There must be at least three anchors. If there were at most two, delete one anchor, or any class if there are none. The remaining $k - 1$ classes would be disjoint and no pair gcd could equal $k - 1$, since that would require two remaining anchors. Every pair gcd would be below $k - 1$, contradicting the least-size choice. This is [Lemma 6\(5\)](#), and justifies beginning both searches with at least three anchors.

If there are **exactly** three anchors, there must be at least two *other* moduli divisible by $d = k - 2$. Here is a direct proof of [Lemma 6\(6\)](#). Retain any one anchor and delete the other two. The remaining $k - 2$ disjoint classes cannot have all pair gcds below $k - 2$; otherwise they form a smaller counterexample. Their gcds are still below k , and a gcd of $k - 1$ is impossible with only one anchor remaining. Hence some two remaining moduli are divisible by d . If no outside modulus is divisible by d , this is impossible. If exactly one outside modulus x is divisible by d , repeating the deletion with each of the three possible retained anchors shows that all three anchors are divisible by d . Two anchors would then have gcd divisible by $\text{lcm}(k - 1, k - 2) = (k - 1)(k - 2) \geq k$, impossible. Thus at least two outside moduli are d -divisible. `special_obstruction` and `extra_good` check exactly that outside count when the final anchor count is three.

2.3 The large-prime rule, including its exceptional case

For $7 \leq k \leq 30$, let $p \geq k/2$ be a prime dividing a modulus. Prime-power support gives at least two p -divisible moduli, and pair-gcd bounds imply $p < k$. Suppose there are $\ell \geq 3$ of them. For each such modulus m_i , let P_i contain its prime divisors other than p . Each P_i is nonempty by the prime-power exclusion. They are pairwise disjoint: a prime q in two of them would give a pair gcd divisible by $pq \geq 2p \geq k$. All these primes are below k by finite normalization.

Each of the other $k - \ell$ moduli meets every anchor in gcd greater than 1. It is not divisible by p , so it shares some prime in each P_i . Choosing one shared prime per anchor gives a signature in $P_1 \times \cdots \times P_\ell$. Equal signatures for two outside moduli would make their gcd divisible by the product of ℓ distinct primes. Because $\ell \geq 3$, this product is at least $2 \cdot 3 \cdot 5 = 30 \geq k$. The signature map is therefore injective, so

$$k - \ell \leq \prod_{i=1}^{\ell} |P_i|. \quad (1)$$

If r is the number of primes below k , then $\sum_i |P_i| \leq r - 1$: p is omitted and the P_i are disjoint. For a fixed positive-integer sum, the product is maximal when sizes differ by at most one; moving one unit from a to b when $a \geq b + 2$ raises the product by $a - b - 1$. Using all $r - 1$ available units gives the upper bounds below. A dash means $\ell > r - 1$, already impossible because each support is nonempty.

r	smallest possible k in $7 \leq k \leq 30$	$\ell = 3$	4	5	6	7	8
3	7	—	—	—	—	—	—
4	8	1	—	—	—	—	—
5	12	2	1	—	—	—	—
6	14	4	2	1	—	—	—
7	18	8	4	2	1	—	—
8	20	12	8	4	2	1	—
9	24	18	16	8	4	2	1
10	30	27	24	16	8	4	2

For rows $r = 3, \dots, 9$, each numeric bound is strictly below $k - \ell$, even using the row's smallest possible k . For $r = 10$, $k = 30$; every entry with $\ell \geq 4$ is below $30 - \ell$. The one equality is $k = 30, \ell = 3$, where (1) forces all three P_i to have size 3, to partition all nine primes other than p , and to realize all 27 signatures on the 27 outside moduli. Of the four primes 17, 19, 23, 29, precisely one can be p , so some $q \geq 17$ belongs to one support. Choose a prime q' in another support; $qq' \geq 34 > 30$. Fix the signature coordinates q, q' . The third coordinate has three choices, each realized by a distinct outside modulus. Any two of those moduli have gcd at least $qq' > 30$, a contradiction. Hence ℓ cannot exceed 2, and support forces $\ell = 2$. This is [Lemma 6\(8\)](#), pp. 5–6, with its final exceptional-case inference made explicit.

The paper's final sentence says two of 17, 19, 23, 29 must lie in separate P_i . That statement does not follow literally when the excluded prime p is one of the four: the other three could occupy one size-three support. The q, q' argument above covers this configuration. `enumerate.py` applies the rule only for $7 \leq k \leq 30$. `clique.py` currently tests `k <= 30` without the lower bound; its published runs begin at $k = 9$, so they are inside the proved range. Its generic strong-mode pruning for $k < 7$ is not justified by this argument.

2.4 Density for every position subset

For any subset S of at least two *positions*, define $M_S = \text{lcm}_{i < j \in S} \text{gcd}(m_i, m_j)$. Modulo M_S , the class $a_i \pmod{m_i}$ occupies exactly $M_S / \text{gcd}(m_i, M_S)$ residue classes. These occupied sets for distinct $i \in S$ must be disjoint: a common residue modulo M_S would make $a_i - a_j$ divisible by $\text{gcd}(m_i, m_j)$, since that gcd divides M_S , and the original two classes would meet. Counting the M_S available residues gives

$$\sum_{i \in S} \frac{M_S}{\text{gcd}(m_i, M_S)} \leq M_S. \quad (2)$$

Thus an exact integer total greater than M_S rejects the tuple, as in [Lemma 5](#) and [Lemma 6\(7\)](#), pp. 2–3. If M is any multiple of M_S , then $\gcd(m_i, M_S) \mid \gcd(m_i, M)$, so each reciprocal term for M is no larger than for M_S . Testing M_S therefore covers every larger permitted period. For a pair, (2) follows automatically from pair gcd at least 2: its two terms total $2/M_S \leq 1$. It suffices for the scripts to enumerate subset sizes at least three. Repeated values still represent different positions, so their multiplicities are retained in the sum.

`density_obstruction`, `density_bad`, and `direct_density_witness` compute (2) with integer lcm, gcd, and division. A bad prefix, triple, or other selected position subset stays in every extension, making each early density rejection hereditary. In `clique.py`, once c copies of a value form a bad prefix or triple, increasing c cannot repair that already-present subset; the loop’s `break` is sound. At a leaf, `first_bad_subset` and `all_subsets_good` enumerate every position subset of size at least three. In particular, rejection does not depend on a floating-point threshold or on a density heuristic.

2.5 Search-path and partial-pool coverage

`enumerate.py` constructs every nondecreasing anchor triple with replacement. For any eligible k -multiset, its first three anchors give exactly one such triple; the remaining occurrences can then be appended in nondecreasing order from `pool`. The initial pool contains every nonanchor compatible with the triple and every compatible anchor no smaller than its third value, **including equal values**. At each extension, `following = pool[i:]` preserves exactly the allowed nondecreasing suffix, intersected with compatibility with the newly chosen value. Thus rejecting `gcd <= 1` or `gcd >= k` at the triple or extension level removes only impossible pairs, and every eligible multiset has a search path. The `--target-length` option is used for diagnostics; strong minimal-counterexample pruning is explicitly forbidden by the code when `target_length != k`.

`clique.py` independently enumerates distinct compatible modulus values, gives each a positive multiplicity, and sorts the result. A repeated value $x \geq k$ is impossible because its pair gcd would be $x \geq k$. A repeated value $x < k$ has multiplicity at most x : for c identical classes, (2) with $M_S = x$ says $c/x \leq 1$. These are the `max_count` bounds. Candidate distinct values form a clique under pair compatibility. Selecting all anchor values first and then all nonanchor values loses no multiset, since order of modulus positions has no mathematical effect. At the anchor stage, `possible_others` contains every nonanchor compatible with the chosen anchors; at the other stage, each recursive `allowed` suffix contains every later distinct compatible value. The capacity rejection

`len(xs) + sum(min(k,x) if x < k else 1 for x in candidates) < k`

uses an **upper** bound on how many positions the remaining values can supply. If even that bound cannot reach k , no completion exists. The analogous anchor-stage inequality with right side 3 only rejects states unable to reach the required three anchors. `expand_others` is called at each anchor state of length at least three, so all possible final anchor counts are covered. Its and `expand_anchors`’s density and triple breaks have the hereditary justification above.

Both strong modes use an overapproximation of future choices, which makes their three no-future rejections safe:

1. **Prime-power support.** If a maximal power q now appears once and no value in the future pool is divisible by q , it can never attain the required second occurrence.
2. **Large-prime count.** For $7 \leq k \leq 30$, more than two current multiples of $p \geq k/2$ cannot be removed. Exactly one current multiple with no future p -multiple cannot reach the required two. (`clique.py` has the lower-bound caveat noted above.)

3. **Exactly three anchors.** If there are exactly three current anchors, no future anchor, fewer than two current **outside** $k - 2$ -multiples, and no future $k - 2$ -multiple, the final tuple violates the proved exact-three rule.

In `enumerate.py`, `pool` is the complete sorted suffix of compatible future values. In `clique.py`, `candidates` is the complete suffix of distinct future values in `expand_others`; at an anchor state, `candidates + possible_others` is a possibly larger-than-real set of future values. Missing support from these pools therefore implies missing support from every actual continuation. The strong partial rules need no estimate of remaining positions; overlooking an impossible branch is harmless, while every branch they reject violates a necessary condition. At complete tuples, `special_obstruction` and `extra_good` apply the same three conditions directly.

2.6 Residue-checking caveat and audit scope

For a fixed modulus tuple, pairwise disjointness depends only on residues modulo $R_i = \text{lcm}_{j \neq i} \gcd(m_i, m_j)$, and $R_i \mid m_i$. Common translation can set the first residue to zero; equal original moduli can be permuted so their residues increase with index. These are sound finite-domain reductions for an exact residue search. A branch is impossible if a proposed residue has pair-gcd divisibility with an assigned one, if an equal-original-modulus order is violated, or if an unassigned variable has no admissible residue. The separately exported trees in `export_refutations.py` and independently replayed trees in `verify_refutations.py` use the equal-original-modulus condition `xs[j] == xs[i]` and check every admissible child and each dead leaf.

An earlier `audit.py` version compared `xs[j]` to the effective bound R_i , rather than to `xs[i]`. Our first audit called this unsound because the original moduli could differ. **That diagnosis was too strong and is withdrawn.** The old comparison also describes a valid compressed symmetry: if $m_j = R_i = q$, then $q \mid m_i$ and every $\gcd(m_i, m_\ell)$ divides q . For every other position ℓ , therefore,

$$\gcd(m_i, m_\ell) = \gcd(q, m_\ell) = \gcd(m_j, m_\ell).$$

Moreover $\gcd(m_i, m_j) = q$ and $R_j = q$. The two finite residue variables have identical bounds and identical constraints with all other variables. They may be exchanged even if their original moduli differ. Within each such effective-modulus group, residues are distinct modulo q and can be sorted by index; this satisfies all comparisons imposed by the old code. Thus we found a mismatch between the documented symmetry and its implementation, not a counterexample to search soundness.

The current code uses the simpler equal-original-modulus comparison, matching the proof above and the independent tree verifier. A fresh run from empty output files reproduced all 107 claims, all 20,436 fact keys and statuses, and every explicit SAT witness unchanged. The minimum verifier checked all 19,704 required smaller SAT facts; the tree verifier replayed all 59 refutations and 40,292 nodes. Both versions have sound symmetry reductions; the package relies on the current version and these independent certificates. Modulus enumeration never calls this residue solver.

3 Appendix B — Complete conditional 24/30 proof

Kevin O'Bryant's [On Z.-W. Sun's Disjoint Congruence Classes Conjecture](https://arxiv.org/abs/math/0604347v2), [arXiv:math/0604347v2](https://arxiv.org/abs/math/0604347v2), Theorem 3 and Lemma 6, states that a **least-size** counterex-

ample cannot have size 24 or 30. The following reconstructs the needed part of Lemma 6, including its finite prime-count argument. It does **not** prove the conjecture independently at either size: the argument assumes the conjecture at every smaller size. No computer search or Lean proof is claimed here.

Suppose, for contradiction, that $k \in \{24, 30\}$ is the least number of pairwise disjoint integer congruence classes $a_i \pmod{m_i}$ with positive moduli and

$$\gcd(m_i, m_j) < k \quad (i \neq j).$$

Among counterexamples of this size, choose one minimizing $\sum_i m_i$. The Chinese remainder criterion says that two classes are disjoint exactly when their pair gcd does **not** divide their residue difference (Proposition 2, p. 2). Consequently every pair gcd is greater than 1: a gcd of 1 would make those two classes meet.

3.1 Normalization and prime support

Put $N = \text{lcm}_{i < j} \gcd(m_i, m_j)$ and $L_k = \text{lcm}(1, \dots, k-1)$. Since every pair gcd is a positive integer below k , $N \mid L_k$. Moreover, each $m_i \mid N$. To see the latter without assuming it, take a maximal prime power $p^e \mid m_i$. If $p^e \nmid N$, every other m_j has p -adic valuation below e . Replacing m_i by m_i/p therefore preserves **every** pair gcd involving i , hence preserves all disjointness conditions by the Chinese remainder criterion. It also lowers $\sum_i m_i$, a contradiction. Applying this to each prime-power factor of m_i proves $m_i \mid N$. Thus every prime dividing any modulus is below k . In fact $N = \text{lcm}_i m_i$, because each pair gcd divides each participating modulus, while every modulus divides N . This is Lemma 6(1), pp. 3–4.

Any prime power $q \mid m_i$ divides N , so by the definition of an lcm it divides at least one pair gcd. Hence it divides at least two moduli; in particular, another modulus besides m_i is divisible by q . This is Lemma 6(3). The moduli cannot be prime powers. Indeed, if $m_i = p^e$, every m_j is divisible by p , since all $\gcd(m_i, m_j) > 1$. Also $p < k$. At least $t = \lceil k/p \rceil$ of the residues have one common value modulo p ; select exactly t such classes. Here $2 \leq t < k$. Subtract their common residue and divide the selected residues and moduli by p . The resulting t classes are pairwise disjoint: for two selected indices, both the residue difference and the pair gcd are divided by p , so nondivisibility is preserved. The conjecture at size $t < k$, available from the least-size assumption, forces a divided pair gcd at least t . Its original pair gcd is then at least $pt \geq k$, a contradiction. This reconstructs Lemma 6(4), pp. 4–5. A modulus 1 is already excluded by the pair-gcd bound, so every modulus divisible by a prime p has another distinct prime factor.

Now let $p = k - 1$, which is respectively 23 or 29 and is prime. At least three moduli are divisible by p . Otherwise delete one of the at most two p -divisible classes (or any class if there are none). The remaining $k - 1$ classes are disjoint. Their pair gcds are below k , and none can equal $k - 1 = p$, because no two remaining moduli are divisible by p . They would be a smaller counterexample. This is the relevant instance of Lemma 6(5), p. 5.

3.2 Counting the possible supports

Write $m_1, \dots, m_\ell$ for **all** the moduli divisible by p , so $\ell \geq 3$, and let P_i be the set of prime divisors of m_i other than p , for $1 \leq i \leq \ell$. Every P_i is nonempty by the prime-power exclusion above. These P_i are pairwise disjoint: if distinct anchors both contained $q \neq p$, their gcd would be divisible by $pq \geq 2(k-1) > k$. Every prime in every P_i is below k , by $m_i \mid N \mid L_k$.

For each outside modulus m_j , $j > \ell$, and each anchor $i \leq \ell$, the pair gcd is greater than 1 but

$p \nmid m_j$. Thus m_i and m_j share some prime in P_i . Choose one such prime $q_i(j)$, for instance the smallest, and assign m_j the signature $(q_1(j), \dots, q_\ell(j)) \in P_1 \times \dots \times P_\ell$. If two outside indices had the same signature, their gcd would be divisible by the product of those ℓ **distinct** primes. As $\ell \geq 3$, that product is at least $2 \cdot 3 \cdot 5 = 30 \geq k$. Hence the signature map is injective, and

$$k - \ell \leq \prod_{i=1}^{\ell} |P_i|. \quad (1)$$

These are the support and injection steps in the proof of [Lemma 6\(8\)](#), pp. 5–6. There are nine primes below 24 and ten below 30. Excluding p , the disjoint nonempty supports have total size at most 8 or 9 respectively. For positive integers $s_i = |P_i|$ of fixed sum, the product is maximized when the sizes differ by at most one: replacing (a, b) with $(a - 1, b + 1)$ when $a \geq b + 2$ increases the product by $a - b - 1 > 0$. Using the entire available prime budget can only increase the maximum. The resulting bounds are:

ℓ	maximum $\prod s_i$, $k = 24$	$24 - \ell$	maximum $\prod s_i$, $k = 30$	$30 - \ell$
3	18	21	27	27
4	16	20	24	26
5	8	19	16	25
6	4	18	8	24
7	2	17	4	23
8	1	16	2	22
9	impossible	—	1	21

For $k = 24$, every row contradicts (1). For $k = 30$, every row except $\ell = 3$ contradicts (1). In that exceptional row, equality throughout is forced: each of the three supports has size 3, together they partition all nine primes 2, 3, 5, 7, 11, 13, 17, 19, 23, and the 27 outside indices realize **every** one of the $3^3 = 27$ signatures.

The prime 17 lies in one support; choose any prime q in a different support. Then $17q \geq 34 > 30$. Fix 17 and q in their two signature coordinates. The third coordinate has three choices, all realized by distinct outside moduli. Any two of those moduli both contain 17 and q , so their gcd is at least $17q > 30$, the final contradiction.

The paper’s last sentence on this exceptional case says that two of 17, 19, 23, 29 must lie in separate P_i . Taken literally for $p = 29$, this does not follow: p is excluded from every P_i , and 17, 19, 23 could occupy the same three-element support. The preceding 17-and- q argument covers that arrangement and supplies the needed conclusion. The paper’s theorem statement is thus supported here by a corrected final counting step, rather than by that particular sentence.